



# Introducción a JAVASCRIPT

(parte 1)

# Contenido



- ❖ ¿Qué es JAVASCRIPT?
- ❖ ES6.
- ❖ Sintaxis.
- ❖ Variables.
- ❖ Tipos de datos.
- ❖ Arrays.
- ❖ Funciones.



# ¿Qué es JavaScript?

Es un lenguaje de programación *interpretado*, *multiparadigma* y de *tipado dinámico*.

- ❖ **Interpretado:** se ejecuta línea por línea, sin ser compilado previamente.
- ❖ **Multiparadigma:** permite combinar distintos paradigmas de programación.
- ❖ **Tipado dinámico:** las variables pueden contener valores de distintos tipos de datos durante el tiempo de ejecución.



# ES6

El estándar **EcmaScript 6** junto a sus versiones posteriores introdujeron importantes mejoras en la sintaxis de **JavaScript**, tales como:

- ❖ Declaración de variables (**let**) y constantes (**const**).
- ❖ Plantillas literales.
- ❖ Funciones de flecha (Arrow functions).
- ❖ Parámetros predeterminados y parámetros **rest**.
- ❖ Operador spread.
- ❖ Promesas - Async / await.
- ❖ Entre otras.



# Sintaxis

La sintaxis de JavaScript es muy similar a la de C, C# y Java.

Las normas básicas son las siguientes:

- ❖ No se tienen en cuenta los espacios en blanco y las nuevas líneas.
- ❖ Se distinguen las mayúsculas y minúsculas.
- ❖ A diferencia de Java, **NO** se define el tipo de las variables.
- ❖ Cada sentencia termina con el carácter de punto y coma (;)
- ❖ Se pueden incluir comentarios (son los mismos que en Java).
- ❖ Utiliza los mismos operadores y estructuras (selectivas e iterativas) que en Java. (\*)



# Operadores

- ❖ Operadores Aritméticos (Ídem C - C# - Java)
- ❖ Operadores de Asignación (Ídem C - C# - Java)
- ❖ Operadores Comparación (Ídem C - C# - Java) (\*)
- ❖ Incremento/Decremento (Ídem C - C# - Java)
- ❖ Operadores Lógicos (Ídem C - C# - Java)



# Declaración de variables

- ❖ Tipado estático o fuertemente tipado (C, C#, Java):
  - Se debe de definir el tipo de dato, obligando a que no pueda haber errores con los mismos.
- ❖ Tipado dinámico o débilmente tipado (JavaScript):
  - No se debe de o tiene porqué especificar el tipo de dato.



# Declaración de variables

En JavaScript existen tres formas de declarar variables:

- ❖ `var`
- ❖ `let`
- ❖ `const`

```
var variable = "hola";           //ámbito global  
  
let otraVariable = 'chau';       //ámbito local  
  
const PI = 3.141592;             //ámbito local  
                                  //pero su valor no puede  
                                  //ser modificado
```

Por convención, las constantes se escriben en mayúsculas y las variables en *camelCase*.

Se aconseja utilizar **let** en lugar de **var**.

Se recomienda el uso de **const** en lugar de **let** para alojar referencias a arrays, objetos, funciones y expresiones regulares.



# Tipos de datos

En JavaScript existen ocho tipos de datos:

- ❖ **number**: para números de cualquier tipo (enteros o flotantes).
- ❖ **bigint**: para números enteros de longitud arbitraria.
- ❖ **string**: para cadenas de cero, uno o varios caracteres. (\*)
- ❖ **boolean**: para valores verdadero o falso (true / false).
- ❖ **null**: para valores desconocidos.
- ❖ **undefined**: para valores no asignados.
- ❖ **symbol**: para identificadores únicos.
- ❖ **object** (no primitivo): para estructuras de datos complejas.

**Nota:** el operador **typeof** permite saber qué tipo de dato se almacena en variables.



# Plantillas literales

Es una nueva forma de construir cadenas literales utilizando back-sticks (``) o tildes invertidos.

Este tipo de plantillas permite utilizar más de una línea sin tener que hacer uso del operador de concatenación (+).

```
let cosa = "algo";
cosa = 10;
cosa = true;

let cadena = "hola";

let otraCadena = 'mundo';

let masCadenas = `con tilde invertido`;

let sinTemplate = cadena + " " + otraCadena + " " + masCadenas + "\n" + cosa + ".";

let template = `concatenando: ${cadena} ${otraCadena} ${masCadenas}. \n${cosa}.`;
```



# Ejemplos



# Arrays (1/2)

Al igual que los objetos, los arrays son colecciones de datos. Se pueden reconocer porque se declaran con corchetes `[]`.

Los arrays son un tipo especial de objetos, por eso no se considera como un tipo de dato más.

Son muy comunes en todo tipo de código.



## Arrays (2/2)

Los arrays disponen de varios métodos muy útiles.

- ❖ **.push()**: Agrega elementos al final de un array.
- ❖ **.pop()**: Elimina el último elemento de un array.
- ❖ **.shift()**: Elimina el primer elemento de un array.
- ❖ **.unshift()**: Agrega elementos al principio de un array.
- ❖ **.join()**: Une los elementos de un array utilizando un separador específico.
- ❖ **.indexOf()**: Retorna el primer índice del elemento a ser buscado o -1.
- ❖ **.includes()**: Retorna **true**, si existe el elemento a ser buscado o **false**, caso contrario.



# Ejemplos



# Funciones

- ❖ Sintaxis de una función básica:
- ❖ Con parámetros predeterminados:
- ❖ Con parámetros REST:

```
function identificador ([args]) {  
    [return;]  
}
```

```
function identificador (param = valor) {  
    [return;]  
}
```

```
function identificador (...params) {  
    [return;]  
}
```



## Fat Arrow (1/2)

El nombre de 'fat arrow' => surge como oposición a las 'flechas finas' ->.

Son funciones similares a las funciones *lambda*, cuyo propósito es:

- ❖ Omitir las palabras **function** y **return**.
- ❖ Implementar el *this* léxico.



## Fat Arrow (2/2)

```
const sumar = (a, b) => { return a + b; }
```

Si la función solo tiene una sentencia **return**, se puede omitir tanto ella como las **llaves**.

```
const doble = (a) => a * 2;
```

Si la función tiene un solo parámetro, pueden omitirse los **paréntesis**.

```
const cinco = () => 5;
```

Si la función no recibe parámetros, de todas formas los **paréntesis** son **obligatorios**.

Si la función tiene más de una sentencia o no retorna valor, las **llaves** son **obligatorias**.

```
const sumar = (a, b) => a + b;
```

```
const doble = a => a * 2;
```

```
const saludar = () => { console.log("Hola"); }
```



# Ejemplos



# Ejercitación